

Generalization in reinforcement learning

Vincent François-Lavet

June 4, 2026

Outline

Motivation for generalization

Generalisation from limited data in supervised learning

Generalisation from limited data in reinforcement learning

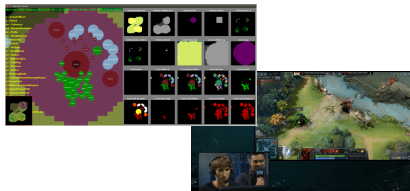
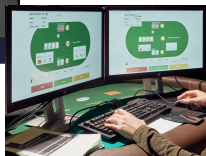
How to improve generalization?

Motivation for tree search techniques and model-based methods

A few additional challenges in (deep) RL

Motivation for generalization

Motivation: Overview



Motivation

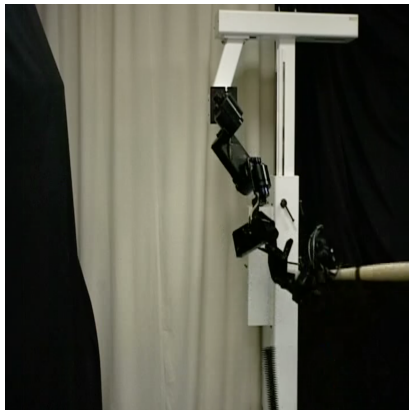


Figure: Application in robotics (credits: Jan Peters'team, Darmstadt)

Challenges of applying RL to real-world problems

In real-world scenarios, it is often not possible to let an agent interact freely and sufficiently in the actual environment:

1. The agent may not be able to interact with the true environment but only with an inaccurate simulation of it. This is known as the **reality gap**.
2. The agent might have access to only **limited data**. This can be due to safety constraints (robotics, medical trials, etc.), compute constraints or due to limited exogenous data (e.g., weather conditions, trading markets).

Challenges of applying RL to real-world problems

In order to deal with the reality gap and limited data, different elements are important:

- ▶ One can aim to develop **a simulator that is as accurate as possible**.
- ▶ One can design the learning algorithm so as to **improve generalization** (and/or use specific transfer learning methods).

Generalization

In an RL algorithm, *generalization* refers to either

- ▶ the capacity to achieve good performance in an environment where limited data has been gathered, or
- ▶ the capacity to obtain good performance in a related environment. This latter case can be tackled with specific *transfer learning* techniques.

Overview

To understand generalization in RL from limited data, we will

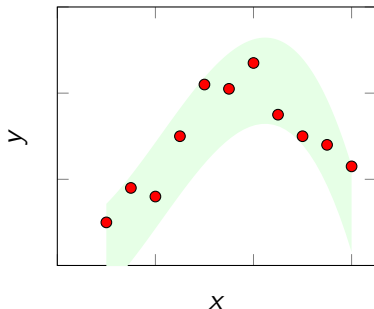
- ▶ recall the concept in supervised learning, and
- ▶ introduce the formulation in RL.

We'll then discuss how an agent can have a good generalization in RL (disclaimer: we'll see where deep RL comes in !)

Generalisation from limited data in supervised learning

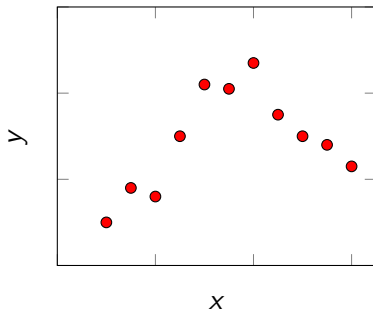
Bias and overfitting in supervised learning

A supervised learning algorithm can be viewed as a mapping from a dataset D_{LS} of learning samples $(x, y) \stackrel{\text{i.i.d.}}{\sim} (X, Y)$ into a predictive model.



Bias and overfitting in supervised learning

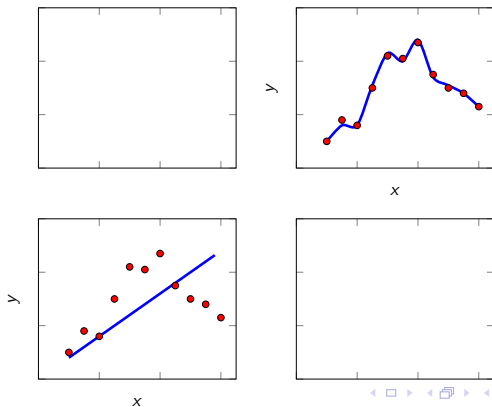
A supervised learning algorithm can be viewed as a mapping from a dataset D_{LS} of learning samples $(x, y) \stackrel{\text{i.i.d.}}{\sim} (X, Y)$ into a predictive model.



You only have access to a limited dataset.

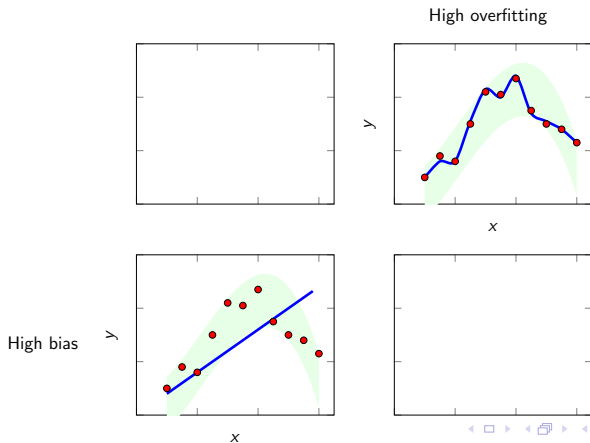
Bias and overfitting in supervised learning

A supervised learning algorithm can be viewed as a mapping from a dataset D_{LS} of learning samples $(x, y) \stackrel{\text{i.i.d.}}{\sim} (X, Y)$ into a predictive model $f(x | D_{LS})$.



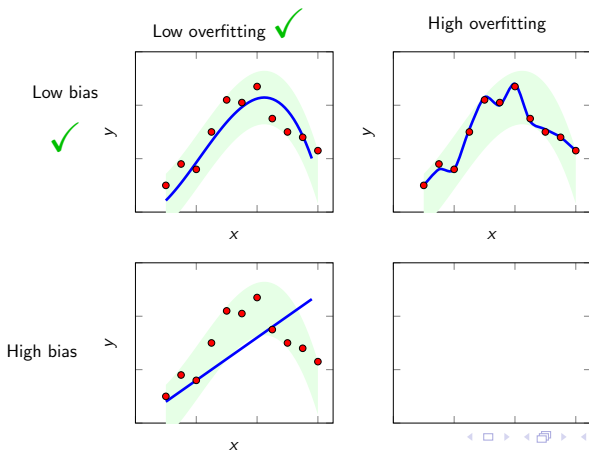
Bias and overfitting in supervised learning

A supervised learning algorithm can be viewed as a mapping from a dataset D_{LS} of learning samples $(x, y) \stackrel{\text{i.i.d.}}{\sim} (X, Y)$ into a predictive model $f(x | D_{LS})$.



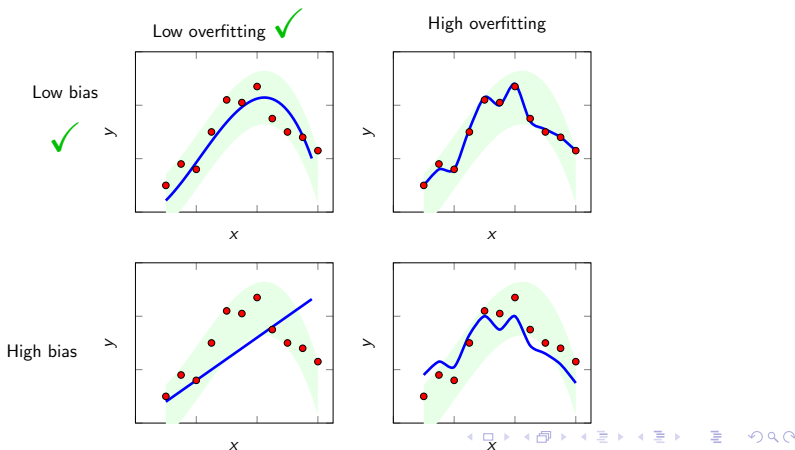
Bias and overfitting in supervised learning

A supervised learning algorithm can be viewed as a mapping from a dataset D_{LS} of learning samples $(x, y) \stackrel{\text{i.i.d.}}{\sim} (X, Y)$ into a predictive model $f(x | D_{LS})$.



Bias and overfitting in supervised learning

A supervised learning algorithm can be viewed as a mapping from a dataset D_{LS} of learning samples $(x, y) \stackrel{\text{i.i.d.}}{\sim} (X, Y)$ into a predictive model $f(x | D_{LS})$.



Bias and overfitting in supervised learning

For one given $x \sim X$, the predictive model $f(x | D_{LS})$ can be illustrated as follows for unseen data $y \sim (Y | X = x)$:

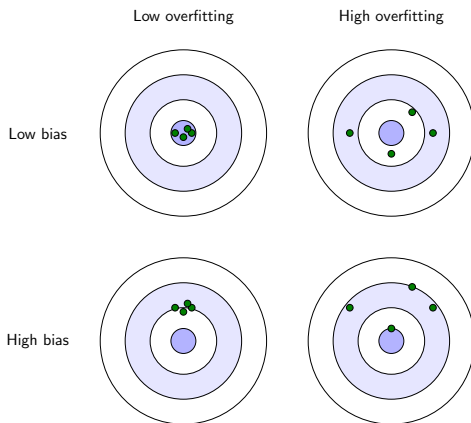
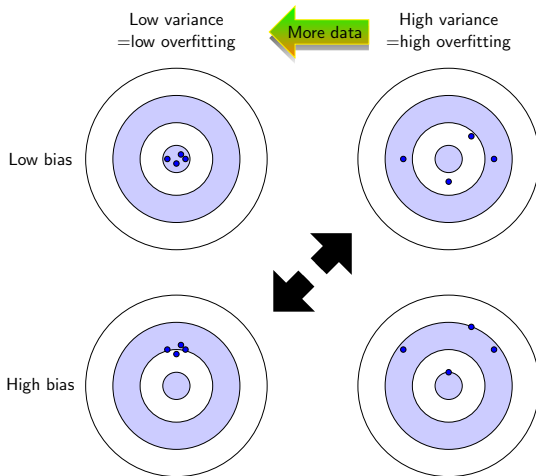


Figure: Illustration of bias and overfitting for unseen tuples, where Y is a 2D continuous RV for visualisation purposes.

Bias and overfitting in supervised learning

There are many choices to optimize the learning algorithm and there is usually a tradeoff between the bias and the overfitting terms to reach best solution.



TRADE OFF BIAS VS VARIANCE

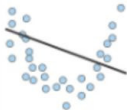
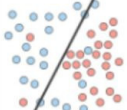
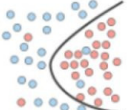
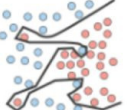
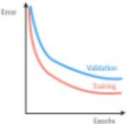
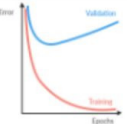
	High Bias	Low Bias Low variance	High variance
characteristics	performs badly training errors= test errors	training errors lower than test	low training error, high test error
Regression			
Classification			
ML (general case)			

Figure: Overview

Bias and overfitting in supervised learning

Assuming a random sampling scheme $D_{LS} \sim \mathcal{D}_{LS}$, $f(x | D_{LS})$ is a random variable, and so is its average error over the input space. The expected value of this quantity is given by:

$$I[f] = \mathbb{E}_X \mathbb{E}_{D_{LS}} \mathbb{E}_{Y|X} L(Y, f(X | D_{LS})), \quad (1)$$

where $L(\cdot, \cdot)$ is the loss function.

Bias and overfitting in supervised learning

Assuming a random sampling scheme $D_{LS} \sim \mathcal{D}_{LS}$, $f(x | D_{LS})$ is a random variable, and so is its average error over the input space. The expected value of this quantity is given by:

$$I[f] = \mathbb{E}_X \mathbb{E}_{D_{LS}} \mathbb{E}_{Y|X} L(Y, f(X | D_{LS})), \quad (1)$$

where $L(\cdot, \cdot)$ is the loss function. If $L(y, \hat{y}) = (y - \hat{y})^2$, the error naturally gives the **bias-variance decomposition**:

$$\mathbb{E}_{D_{LS}} \mathbb{E}_{Y|X} (Y - f(X | D_{LS}))^2 = \sigma^2(x) + \text{bias}^2(x), \quad (2)$$

where

$$\begin{aligned} \text{bias}^2(x) &\triangleq (\mathbb{E}_{Y|x}(Y) - \mathbb{E}_{D_{LS}} f(x | D_{LS}))^2, \\ \sigma^2(x) &\triangleq \underbrace{\mathbb{E}_{Y|x} (Y - \mathbb{E}_{Y|x}(Y))^2}_{\text{Internal variance}} + \underbrace{\mathbb{E}_{D_{LS}} (f(x | D_{LS}) - \mathbb{E}_{D_{LS}} f(x | D_{LS}))^2}_{\text{Parametric variance} = \text{overfitting}}. \end{aligned}$$

Bias and overfitting in reinforcement learning

This bias-variance decomposition highlights a tradeoff between

- ▶ an error directly introduced by the learning algorithm (the bias) and
- ▶ an error due to the limited amount of data available (the parametric variance).

Bias and overfitting in reinforcement learning

This bias-variance decomposition highlights a tradeoff between

- ▶ an error directly introduced by the learning algorithm (the bias) and
- ▶ an error due to the limited amount of data available (the parametric variance).

Note that there is no such direct bias-variance decomposition for loss functions other than the L_2 loss! It is however always possible to decompose the prediction error with a term related to the lack of expressivity of the model (the bias) and a term due to the limited amount of data (overfitting comes from the variance of $f(x | D_{LS})$ on the loss when $D_{LS} \sim \mathcal{D}_{LS}$ but $\neq$ statistical variance if loss function is not L_2).

Questions so far?

Next: generalization in reinforcement learning

Generalisation from limited data in reinforcement learning

Bias and overfitting in supervised learning

Since there is no direct bias-variance decomposition for loss functions other than L_2 loss in supervised learning, there is not an actual "bias-variance" tradeoff in RL.

However, there is still a tradeoff between a sufficiently rich learning algorithm (to reduce the model bias, which is present even when the amount of data would be unlimited) and a learning algorithm not too complex (so as to avoid overfitting to the limited amount of data).

Bias and overfitting in RL

The *batch* or *offline* algorithm in RL can be seen as mapping a dataset $D \sim \mathcal{D}$ into a policy π_D (independently of whether the policy comes from a model-based or a model-free approach):

$$D \rightarrow \pi_D.$$

In an MDP, the suboptimality of the expected return can be decomposed as follows:

$$\begin{aligned} \mathbb{E}_{D \sim \mathcal{D}} [V^{\pi^*}(x) - V^{\pi_D}(x)] &= \underbrace{(V^{\pi^*}(x) - V^{\pi_{D\infty}}(x))}_{\text{asymptotic bias}} \\ &+ \underbrace{\mathbb{E}_{D \sim \mathcal{D}} [(V^{\pi_{D\infty}}(x) - V^{\pi_D}(x))]}_{\text{error due to finite size of the dataset } D_s \text{ referred to as } \textit{overfitting}}]. \end{aligned}$$

How to obtain the best policy?

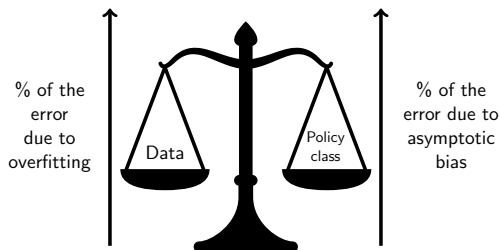


Figure: Schematic representation of the bias-overfitting tradeoff.

How to improve generalization?

How to improve generalization?

We can optimize the bias-overfitting tradeoff thanks to the following elements:

- ▶ an **abstract representation** that discards non-essential features,
- ▶ the **objective function** (e.g., reward shaping, tuning the training discount factor) and
- ▶ the **learning algorithm** (type of function approximator and model-free vs model-based).

And of course, if possible:

- ▶ improve the dataset (exploration/exploitation dilemma in an online setting)

1. Abstract representation

The appropriate level of abstraction plays a key role in the bias-overfitting tradeoff.

- ▶ When considering many features on which to base the policy, an RL algorithm may take into consideration spurious correlations, which leads to overfitting.
- ▶ Removing features that discriminate states with a very different role in the dynamics introduces a bias.

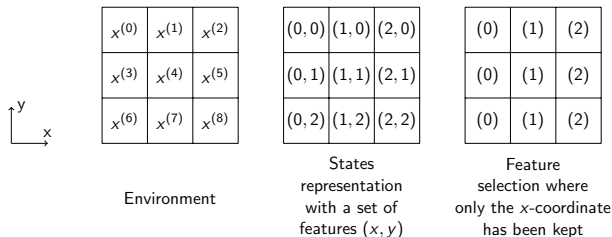


Figure: Illustration of the abstract representation.

1. Abstract representation

In POMDPs, policies are based on a state representation built from histories of observations, actions and rewards.

With $\mathcal{H}_t = \Omega \times (\mathcal{A} \times \mathcal{R} \times \Omega)^t$, $\mathcal{H} = \bigcup_{t=0}^{\infty} \mathcal{H}_t$, we consider a mapping $\phi : \mathcal{H} \rightarrow \phi(\mathcal{H})$, where $\phi(\mathcal{H}) = \{\phi(H) | H \in \mathcal{H}\}$.

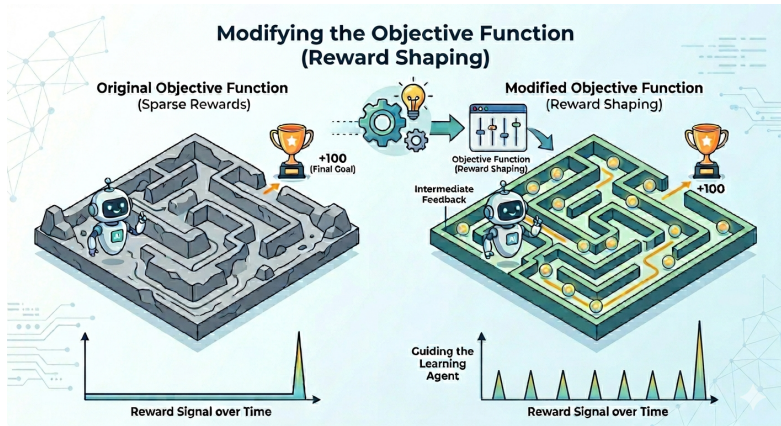
- ▶ On the one hand, a mapping ϕ with a low cardinality $|\phi(\mathcal{H})|$ reduces the risk of **overfitting** ($|\Pi_{\phi(\mathcal{H})}| \leq |\mathcal{A}|^{|\phi(\mathcal{H})|}$).
- ▶ On the other hand, when ϕ discards information from the history, the state representation $\phi(H)$ might depart from sufficient statistics, which creates an **asymptotic bias**.

2. Modifying the objective function

In order to improve the policy learned by a deep RL algorithm, one can optimize an objective function that diverts from the actual objective. By doing so, a bias is usually introduced but this can in some cases help with generalization. The main approaches to modify the objective function are either

- ▶ to modify the reward of the task to ease learning (reward shaping), or
- ▶ tune the discount factor at training time.

2. Modifying the objective function (reward shaping)



2. Modifying the objective function (reward shaping)

Idea: Modify the reward signal to guide learning, ideally without changing the optimal policy.

Reward shaping:

$$r'_t = r_t + F(s_t, s_{t+1})$$

where F is a shaping function that injects additional learning signal.

Key condition (policy invariance):

$$F(s, s') = \gamma\Phi(s') - \Phi(s)$$

- ▶ $\Phi(s)$: potential function over states
- ▶ Ensures optimal policy is preserved

Intuition:

- ▶ Accelerates learning by providing denser feedback
- ▶ Helps exploration in sparse-reward environments

2. Modifying the objective function (discount factor)

Motivation from neurosciences:

- ▶ *Empirical studies of cognitive mechanisms in delay of gratification*: The capacity to wait longer for the preferred rewards seems to develop markedly only at about ages 3-4 (“marshmallow experiment”).

In the online setting, the optimal tradeoff evolves as more data is obtained. There is for instance an interest of increasing the discount factor when more data of interest is gathered.

2. Modifying the objective function (discount factor)

Increasing the discount factor (e.g., in the DQN algorithm) from 0.95 to 0.99 through learning can significantly improve learning.

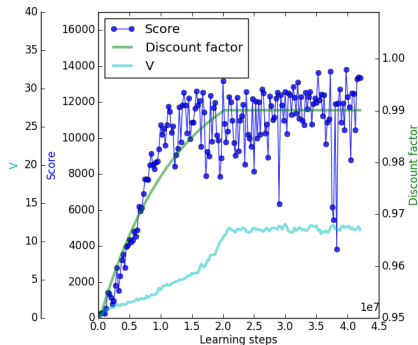
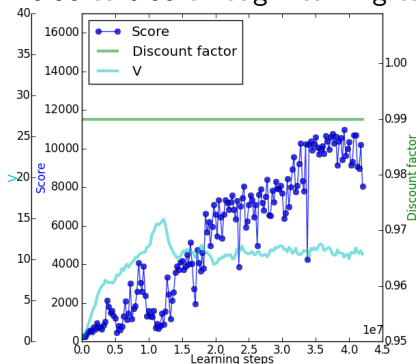
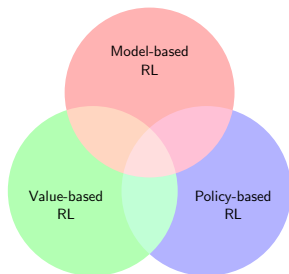
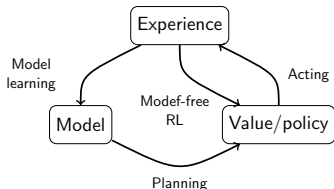


Figure: Illustration for the game q-bert of a discount factor γ held fixed on the left and an adaptive discount factor on the right.

3. Choice of the learning algorithm and function approximator selection

In general, an RL agent may include one or more of the following components:

- ▶ a representation of a value function that provides a prediction of how good is each state or each couple state/action,
- ▶ a direct representation of the policy $\pi(x)$ or $\pi(x, a)$, or
- ▶ a model of the environment in conjunction with a planning algorithm.



Deep learning has brought its generalization capabilities to RL. ▶

3. Choice of the learning algorithm and function approximator selection

- ▶ The function approximator in deep learning characterizes how the features will be treated into higher levels of abstraction. A fortiori, it is related to feature selections (e.g., an attention mechanism), etc.
- ▶ Depending on the task, finding a performant function approximator is easier in either a model-free or a model-based approach. The choice of relying more on one or the other approach is thus also a crucial element to improve generalization.

3. Choice of the learning algorithm

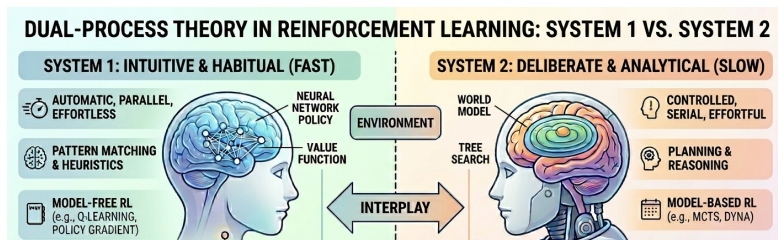
The respective strengths of the model-free versus model-based approaches depend on different factors.

- ▶ If the agent does not have access to a generative model of the environment, the learned model will have some inaccuracies.
- ▶ Second, a model-based approach requires working in conjunction with a planning algorithm, which is often computationally demanding.
- ▶ Third, for some tasks, the model of the environment may be learned more efficiently due to the particular structure of the task.

3. Choice of the learning algorithm: a parallel with neurosciences

In cognitive science, there is a dichotomy between two modes of thoughts (*D. Kahneman. (2011). Thinking, Fast and Slow*) :

- ▶ a "System 1" that is fast and instinctive and
- ▶ a "System 2" that is slower and more logical.



Motivation for tree search techniques and model-based methods

Motivation for planning with tree search

Given that you're playing the crosses, what would be your next move?

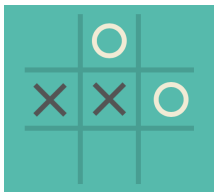


Figure: Illustration of a state in the tic-tac-toe game.

→ how did you come up with that choice?

Motivation

MCTS algorithms need (only) a generative model of the environment (i.e. model-based):

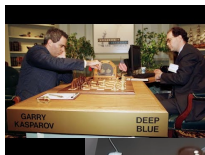
$$x', r \sim G(x, a)$$

Advantages:

- ▶ it is possible to obtain samples without having the whole transition function for the model in an explicit form.
- ▶ it can learn a strong policy only where needed (from the current state x).
- ▶ it is useful for a sequence of decisions.

Motivation

- ▶ Tree search algorithms can be used along with different heuristics as well as model-free deep RL techniques.
- MCTS has been a key part of alpha Go for instance.



Monte-Carlo Tree Search methods

The overall idea is to estimate the action with the highest expected return.

$$V^*(x) = Q^*(x, a = \pi^*) = \mathbb{E}_{\pi^*}[r_0 + \gamma r_1 + \dots]$$

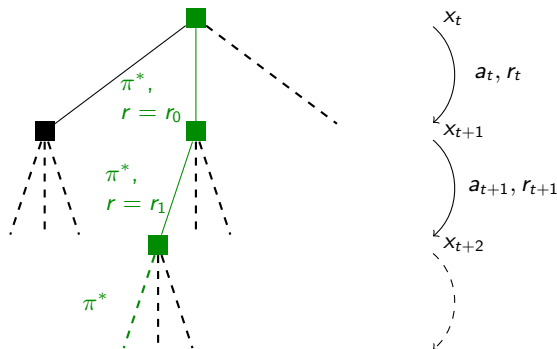


Figure: Illustration of model-based planning with tree search.

Monte-Carlo Tree Search methods

Monte-Carlo Tree Search (MCTS) is a key algorithm for finding a good policy trying out trajectories from a state x in an MDP.

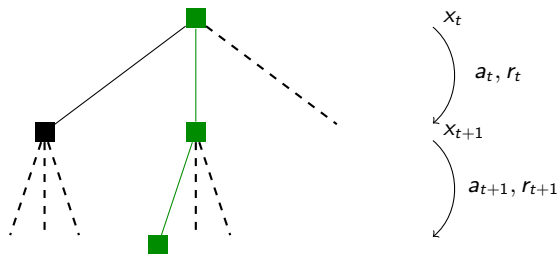
It is a combination of

- ▶ Monte-Carlo methods, i.e. a class of computational algorithms that rely on repeated random sampling to obtain numerical results.
- ▶ Tree search, i.e. methods that allow exploring a tree data structure.

Purely random Monte Carlo tree search

1. Build a search tree with the current state of the agent at the root
2. Until convergence
 - ▶ Sample a trajectory with random actions (e.g. actions distributed uniformly)
 - ▶ Update estimates of the value functions where appropriate using already simulated episodes by backing up the values from the leafs of the tree to the root node:

$$Q(x, a) = \frac{\sum_{x'} n(x' | x, a) (r + \gamma \max_{a' \in \mathcal{A}} Q(x', a'))}{n(x, a)}. \quad (3)$$



Monte Carlo tree search in n-players games

Monte Carlo tree search can be combined with the “*minimax*” / “*maximin*” approach. The *maximin* approach is a decision rule for maximizing the rewards for a worst case (maximum loss) scenario.

→ well-suited for 2-players zero-sum games (or n-players)!

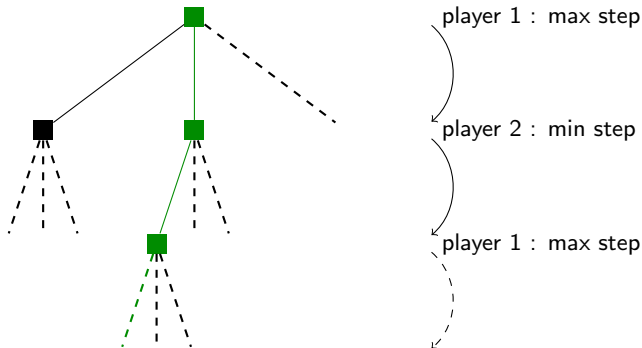


Figure: Illustration of tree search in 2-players zero-sum game.

Purely random Monte Carlo tree search

1. Build a search tree with the current state of the agent at the root
2. Until convergence
 - ▶ Sample a trajectory with random actions (e.g. actions distributed uniformly)
 - ▶ Update estimates of the value functions where appropriate using already simulated episodes by backing up the values from the leafs of the tree to the root node:

$$Q(x,a,\cdot) = \frac{\sum_{x'} n(x'|x,a) \left(r + \gamma \max_{a(p1) \in \mathcal{A}_{p1}} \min_{a(p2) \in \mathcal{A}_{p2}} Q(x',\cdot,\cdot) \right)}{n(x,a)}$$

A few additional challenges in (deep) RL

Another important challenge: transfer learning



Figure: Transfer learning between different renderings. Picture from *"Playing for Data: Ground Truth from Computer Games"*, Richter, S. and Vineet, V., et al

Another important challenge: exploration

- ▶ Undirected exploration (e.g. ϵ -greedy)
- ▶ Directed exploration with an estimate of “novelty”
 - ▶ when rewards are not sparse, a measure of the uncertainty on the value function can be used,
 - ▶ if sparse rewards or no rewards, some exploration rewards have to be used.

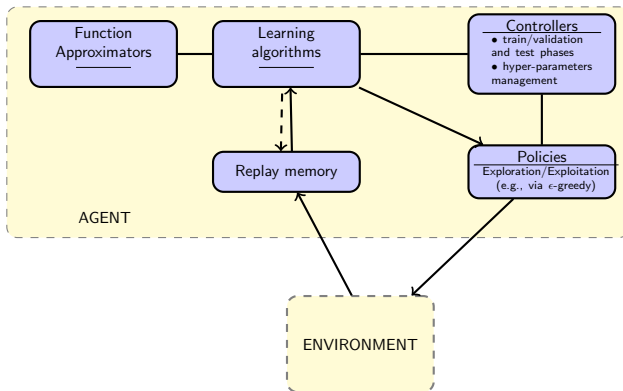
Combining model-based and model-free via abstract representations

Why are we interested in learning everything through one abstract representation?

- ▶ it can help enforce a good generalization,
- ▶ planning is computationally efficient,
- ▶ it facilitates interpretation of the decisions taken by the agent,
- ▶ it enables strategies for transfer learning, and
- ▶ it can be used to improve exploration.

Conclusion

The different components of the RL agent play a role in performance.



Conclusion

Generalization is a central concept in the field of machine learning, and reinforcement learning is no exception.

In this lecture we have seen

- ▶ what generalization is in RL,
- ▶ how an agent can have a good generalization,
- ▶ a few words about model-based RL
- ▶ some broader challenges in RL

More information can be found in the following book:

V François-Lavet, et al. "*An introduction to deep reinforcement learning*". Foundations and Trends in ML.

<https://arxiv.org/abs/1811.12560>

Questions?

Next two weeks outlook

Next weeks, we'll see how we can learn from data an approximation of key functions (with and without deep learning):

- ▶ The action value function $Q^*(x, a)$
- ▶ The policy $\pi^*(x, a)$

Project

Based on the game connect 4, how can you develop an MCTS tree search algorithm with the UCT rule for exploration/exploitation?

- ▶ Create the environment along with the terminal conditions. Reward is 1 for the winner, -1 for the loser and 0 for draw.
- ▶ The opponent is purely random at all times, the play of the opponent can be seen as part of the transition function (no need of the max/min algorithm).
- ▶ You play the green circles.

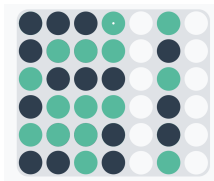


Figure: “connect 4” game, green to play.

- ▶ Bonus of 1 point if you investigate in an interesting way the case from an empty connect 4 game

Project

Your need to provide:

- ▶ A report (limit of 2 pages / approximately 1000 words)
 - ▶ a description of the main parts of your algorithm (4 points)
 - ▶ vizualisation of at least one game (with insight about some winning probabilities for the actions) (3 points)
 - ▶ provide meaningful information about the convergence process (3 points)
- ▶ You should provide the source code of your scripts (python). You are allowed to use existing code (not necessarily easier!) as long as you mention the source explicitly both in the code and in the report.
- ▶ Groups of 2
- ▶ Deadline: Dec 8 23:59.

Hints for the project

Here are the first things you need to do:

- ▶ A way to encode the state of the game.
- ▶ You need to “code” the conditions for a checking winning/losing position.
- ▶ Build a tree structure based on the parent-child relation between two objects.

Questions?